SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY

MUKKA, MANGALURU – 574146

MICROSERVICES ARCHITECTURE & IMPLEMENTATION

A MINI PROJECT REPORT

ON

HOSPITAL MANAGEMENT SYSTEM

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN COMPUTER SCIENCE AND ENGINEERING

Submitted By

KEERTHANA L S 01SU23CS088

RITHIKSHA 01SU23CS151

Under the guidance of

Prof. YOGESH
Dept. of CSE

2025 - 2026

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Objectives
4. Technologies Used
5. System Architecture
6. Working Flow
7. Database Design
8. Features
9. Implementation Details
10. API End Points
11. User Interface
12. Security Mechanisms
13. Testing
14. Advantages and Limitations
15. Future Enhancement / Future Scope
16. Conclusion

Hospital Management System Report

ABSTRACT

The Hospital Management System (HMS) developed using Microservices Architecture is designed to provide an efficient, scalable, and flexible solution for managing hospital operations. In traditional monolithic systems, all functionalities such as patient management, doctor scheduling, appointments, and billing are tightly coupled, making the application difficult to maintain, update, and scale.

To overcome these limitations, this project follows a microservices-based approach, where each major functionality is developed as an independent service. The system is divided into multiple microservices such as Patient Service, Doctor Service, Appointment Service, and Billing Service. Each service works independently and communicates with other services through RESTful APIs.

This modular architecture improves system maintainability, fault isolation, and scalability. It allows each service to be developed, tested, deployed, and scaled independently without affecting the complete system. As a result, the Hospital Management System becomes more reliable, flexible, and suitable for modern healthcare applications.

1. INTRODUCTION

A Hospital Management System (HMS) is a software application developed to manage and streamline the day-to-day activities of a hospital, including patient records, doctor scheduling, appointments, billing, and administrative tasks. Traditionally, such systems are built as monolithic applications, where all modules are combined into one large system. Although functional, monolithic systems are difficult to scale, maintain, and update as the application grows.

To solve these issues, modern software systems use Microservices Architecture, where the application is divided into multiple small and independent services. Each service handles one specific business function and operates independently. For example, patient details are handled by the Patient Service, appointments by the Appointment Service, and billing by the Billing Service.

In this project, the Hospital Management System is developed using a microservices-based approach to ensure better scalability, flexibility, fault isolation, and ease of deployment. This architecture also supports modern technologies such as Docker, REST APIs, cloud deployment, and distributed databases. It provides a more efficient and future-ready solution for healthcare institutions.

2. OBJECTIVES

• Improve the efficiency of hospital operations.
• Divide the system into independent and manageable microservices.
• Provide scalability so that services can grow independently.
• Allow easier updates and maintenance.
• Ensure fault isolation, so failure in one service does not affect others.
• Support faster development and deployment.
• Allow the use of different technologies for different services.
• Improve patient, doctor, and appointment data management.
• Enhance the user experience with a responsive and structured interface.
• Enable integration with future healthcare technologies and systems.

3. TECHNOLOGIES USED

Frontend Technologies:
• HTML
• CSS
• JavaScript

Backend Technologies:
• Node.js / Java (Spring Boot)

Database Technologies:
• MySQL / MongoDB

API Communication:
• REST APIs

Deployment / Containerization:
• Docker

Development Tools:
• VS Code / IntelliJ / Eclipse
• Postman
• Git / GitHub

4. SYSTEM ARCHITECTURE

The Hospital Management System follows a Microservices Architecture.

Architecture Components:
• Client Layer – Users such as patients, doctors, and administrators access the system through a web or mobile interface.
• API Gateway – Acts as the single entry point for all requests from the client side.
• Microservices Layer – Patient Service, Doctor Service, Appointment Service, and Billing Service.
• Database Layer – Each service has its own separate database.

Architecture Flow:
Client (User) → API Gateway → Microservices → Separate Databases

This architecture makes the system scalable, flexible, reliable, and easy to maintain.

5. WORKING FLOW

1. User Access – A patient, doctor, or admin accesses the system through a web or mobile interface.
2. API Gateway – All incoming requests are first sent to the API Gateway.
3. Service Handling – The API Gateway identifies which microservice is required and forwards the request.
4. Microservice Processing:
   • Patient Service → Manages patient registration and records.
   • Doctor Service → Manages doctor details and schedules.
   • Appointment Service → Handles appointment booking and scheduling.
   • Billing Service → Handles billing and payment information.
5. Inter-Service Communication – If needed, services communicate with one another through REST APIs.
6. Database Access – Each service stores or retrieves data from its own dedicated database.
7. Response Back – The final response is sent back to the user interface.

6. DATABASE DESIGN

In the microservices-based Hospital Management System, each microservice has its own separate database.

Database Distribution:
• Patient Service → Patient Database
• Doctor Service → Doctor Database
• Appointment Service → Appointment Database
• Billing Service → Billing Database

Example Entities:
Patient Database:
• Patient ID, Name, Age, Gender, Address, Contact Number, Medical History

Doctor Database:
• Doctor ID, Name, Specialization, Availability, Contact Information

Appointment Database:
• Appointment ID, Patient ID, Doctor ID, Date, Time, Status

Billing Database:
• Bill ID, Patient ID, Service Charges, Payment Status, Payment Date

7. FEATURES

• Patient registration and record management
• Doctor management and scheduling
• Appointment booking and tracking
• Billing and payment management
• Independent microservices for each module
• Fast and scalable system performance
• Secure handling of user and hospital data
• Easy maintenance and modular development
• Separate databases for each service
• API-based communication between services

8. IMPLEMENTATION DETAILS

The implementation of the Hospital Management System is based on microservices architecture, where each functional module is designed as an independent service.

Implementation Highlights:
• Frontend developed using HTML, CSS, and JavaScript
• Backend built using Node.js or Java Spring Boot
• Patient, Doctor, Appointment, and Billing modules are separate microservices
• Each microservice exposes its own REST API endpoints
• Services communicate using API calls
• Each service has its own dedicated database
• Docker containers are used for deployment
• The application can be scaled independently for each service

9. API END POINTS

Patient Service:
• GET /patients
• GET /patients/{id}
• POST /patients
• PUT /patients/{id}
• DELETE /patients/{id}

Doctor Service:
• GET /doctors
• GET /doctors/{id}
• POST /doctors
• PUT /doctors/{id}
• DELETE /doctors/{id}

Appointment Service:
• GET /appointments
• POST /appointments/book
• PUT /appointments/{id}
• DELETE /appointments/{id}

Billing Service:
• GET /billing
• POST /billing/pay

10. USER INTERFACE

Patient Interface:
• Register and login
• View personal details
• Book appointments
• View appointment history and records

Doctor Interface:
• Login and access dashboard
• View appointment schedule
• Manage patient details and records

Admin Interface:
• Manage system users
• Monitor hospital operations
• Manage doctors, appointments, and billing

UI Characteristics:
• Simple and user-friendly design
• Easy navigation between modules
• Clean layout for all user roles
• Responsive design for better usability

11. SECURITY MECHANISMS

• User Authentication – Login system to verify users
• Authorization – Role-based access for patient, doctor, and admin
• JWT Token Security – Used to securely manage user sessions
• Secure APIs (HTTPS) – Protects communication between client and server
• Data Encryption – Protects confidential data during storage and transfer

12. TESTING

• Unit Testing – Each microservice is tested individually
• Integration Testing – Ensures proper communication between services
• API Testing – REST APIs are tested using Postman
• Performance Testing – Checks system behavior under load
• Security Testing – Identifies vulnerabilities in authentication and APIs

13. ADVANTAGES AND LIMITATIONS

Advantages:
• Easy to scale individual services
• Independent development and deployment
• Faster updates and maintenance
• Better fault isolation
• Improved flexibility and modularity
• Supports modern cloud-based deployment

Limitations:
• More complex to manage than monolithic systems
• Increased network communication between services
• Data consistency can be challenging
• Higher deployment and monitoring cost
• Requires proper orchestration and service management

14. FUTURE ENHANCEMENT / FUTURE SCOPE

• AI-based diagnosis and treatment recommendations
• Telemedicine / online consultation support
• Mobile application integration
• Cloud-based deployment and scaling
• Advanced reporting and data analytics
• Real-time notifications via SMS or Email
• Online payment gateway integration
• Electronic Health Record (EHR) integration
• Chatbot support for patients

15. CONCLUSION

The Hospital Management System using Microservices Architecture provides a modern, scalable, and flexible solution for managing hospital operations. By dividing the application into independent services such as patient management, doctor management, appointment handling, and billing, the system becomes easier to develop, maintain, deploy, and scale.

This architecture improves system reliability, performance, and fault tolerance while also supporting future expansion. Overall, the project demonstrates how microservices can be effectively used to build a secure, efficient, and high-quality healthcare management platform.